

LEVEL 15 — Pandas String Operations

A line-by-line practical guide to Pandas Series string methods, with code and expected output.

1. Basic setup

```
import pandas as pd

names = pd.Series(["  alice smith  ", "BOB JONES", "Charlie Brown", "david lee"])
```

Output:

```
0      alice smith
1      BOB JONES
2    Charlie Brown
3      david lee
dtype: object
```

Important: Pandas string methods are normally accessed through `.str`, for example `names.str.lower()`.

1. `.str` — String accessor

The `.str` accessor tells Pandas that we want to perform a string operation on each value of the Series.

Code — line by line:

```
s = pd.Series(["Python", "Pandas", "NumPy"])

print(s.str.lower())
```

Output:

```
0    python
1    pandas
2     numpy
dtype: object
```

2. `lower()`

Converts letters to lowercase.

Code — line by line:

```
s = pd.Series(["HELLO", "Python", "PANDAS"])

result = s.str.lower()
print(result)
```

Output:

```
0    hello
1    python
2    pandas
dtype: object
```

3. `upper()`

Converts letters to uppercase.

Code — line by line:

```
s = pd.Series(["hello", "Python", "pandas"])

result = s.str.upper()
print(result)
```

Output:

```
0    HELLO
1    PYTHON
2    PANDAS
dtype: object
```

4. title()

Capitalizes the first letter of each word.

Code — line by line:

```
s = pd.Series(["hello world", "python pandas", "data science"])

result = s.str.title()
print(result)
```

Output:

```
0      Hello World
1    Python Pandas
2    Data Science
dtype: object
```

5. capitalize()

Capitalizes the first character and makes the remaining characters lowercase.

Code — line by line:

```
s = pd.Series(["hello world", "PYTHON", "data SCIENCE"])

result = s.str.capitalize()
print(result)
```

Output:

```
0      Hello world
1      Python
2    Data science
dtype: object
```

6. strip()

Removes whitespace from both the beginning and end.

Code — line by line:

```
s = pd.Series([" hello ", " python ", "pandas"])

result = s.str.strip()
print(result)
```

Output:

```
0      hello
1     python
2     pandas
dtype: object
```

7. lstrip()

Removes whitespace from the left side only.

Code — line by line:

```
s = pd.Series([" hello", "  python", "pandas"])

result = s.str.lstrip()
print(result)
```

Output:

```
0      hello
1     python
2     pandas
dtype: object
```

8. rstrip()

Removes whitespace from the right side only.

Code — line by line:

```
s = pd.Series(["hello ", "python ", "pandas"])

result = s.str.rstrip()
print(result)
```

Output:

```
0    hello
1   python
2   pandas
dtype: object
```

9. replace()

Replaces matching text. With `regex=False`, the search is treated as ordinary text.

Code — line by line:

```
s = pd.Series(["I like Python", "Python is easy", "Learn Python"])

result = s.str.replace("Python", "Pandas", regex=False)
print(result)
```

Output:

```
0    I like Pandas
1   Pandas is easy
2   Learn Pandas
dtype: object
```

10. contains()

Checks whether the text contains the specified pattern and returns True or False.

Code — line by line:

```
s = pd.Series(["Python", "Pandas", "NumPy", "Python Pandas"])

result = s.str.contains("Python", na=False)
print(result)
```

Output:

```
0    True
1   False
2   False
3    True
dtype: bool
```

11. startswith()

Checks whether each string starts with the specified text.

Code — line by line:

```
s = pd.Series(["Python", "Pandas", "NumPy", "Python Basics"])

result = s.str.startswith("Py")
print(result)
```

Output:

```
0    True
1   False
2   False
3    True
dtype: bool
```

12. endswith()

Checks whether each string ends with the specified text.

Code — line by line:

```
s = pd.Series(["hello.py", "data.csv", "notes.txt", "model.py"])
```

```
result = s.str.endswith(".py")
print(result)
```

Output:

```
0      True
1     False
2     False
3      True
dtype: bool
```

13. find()

Returns the position of the first occurrence. If the text is not found, it returns -1. Positions start at 0.

Code — line by line:

```
s = pd.Series(["Python", "Pandas", "Data Science"])
result = s.str.find("a")
print(result)
```

Output:

```
0    -1
1     1
2     1
dtype: int64
```

14. len()

Returns the number of characters in each string, including spaces.

Code — line by line:

```
s = pd.Series(["Python", "Pandas", "Data Science"])
result = s.str.len()
print(result)
```

Output:

```
0      6
1      6
2     12
dtype: int64
```

15. split()

Splits each string into a list using the specified separator.

Code — line by line:

```
s = pd.Series(["Alice Smith", "Bob Jones", "Charlie Brown"])
result = s.str.split(" ")
print(result)
```

Output:

```
0      [Alice, Smith]
1      [Bob, Jones]
2    [Charlie, Brown]
dtype: object
```

16. join()

Joins the elements of each list using the separator.

Code — line by line:

```
s = pd.Series(["Python", "Pandas"], ["Data", "Science"])
result = s.str.join(" - ")
print(result)
```

Output:

```
0 Python - Pandas
1 Data - Science
dtype: object
```

17. extract()

Extracts groups from text using a regular expression. Parentheses create capture groups.

Code — line by line:

```
s = pd.Series([
    "Name: Alice, Age: 25",
    "Name: Bob, Age: 30"
])

result = s.str.extract(r"Name: (\w+), Age: (\d+)")
print(result)
```

Output:

```
      0      1
0  Alice  25
1   Bob  30
```

18. Regular expressions with Pandas

Regular expressions (regex) let you describe text patterns. Here, the pattern checks for a basic email-like structure.

Code — line by line:

```
s = pd.Series([
    "alice@gmail.com",
    "bob@yahoo.com",
    "charlie@gmail.com",
    "invalid-email"
])

result = s.str.contains(r"^[a-zA-Z0-9]+@[a-zA-Z0-9]+\.[a-zA-Z]+$", regex=True, na=False)
print(result)
```

Output:

```
0    True
1    True
2    True
3   False
dtype: bool
```

Useful String Operations — Quick Reference

Operation	Purpose	Example
.str	Access string methods	s.str.lower()
lower()	Lowercase	s.str.lower()
upper()	Uppercase	s.str.upper()
title()	Title case	s.str.title()
capitalize()	First character uppercase	s.str.capitalize()
strip()	Remove both-side whitespace	s.str.strip()
lstrip()	Remove left whitespace	s.str.lstrip()
rstrip()	Remove right whitespace	s.str.rstrip()
replace()	Replace text	s.str.replace('a','b')
contains()	Check for pattern/text	s.str.contains('Py')
startswith()	Check beginning	s.str.startswith('Py')
endswith()	Check ending	s.str.endswith('.py')
find()	Find first position	s.str.find('a')
len()	String length	s.str.len()
split()	Split into list	s.str.split(' ')
join()	Join list elements	s.str.join('-')
extract()	Extract regex groups	s.str.extract(r'(\d+)')
Regex	Match complex patterns	s.str.contains(r'^A', regex=True)

Important Notes

- Most Pandas string operations are used as `series.str.method()`.
- String methods can handle missing values, but the exact result may depend on the method and parameters. For Boolean methods such as `contains()`, using `na=False` is often convenient when filtering.
- In `extract()`, parentheses such as `(w+)` create capture groups that become columns.
- Regex is especially useful for emails, phone numbers, IDs, dates, extracting numbers, and filtering text patterns.
- For literal replacement, `regex=False` avoids treating special regex characters as patterns.

Practice tip: For every method, first create a small Series, run the method, and inspect the output. Then try the same method on a DataFrame column.